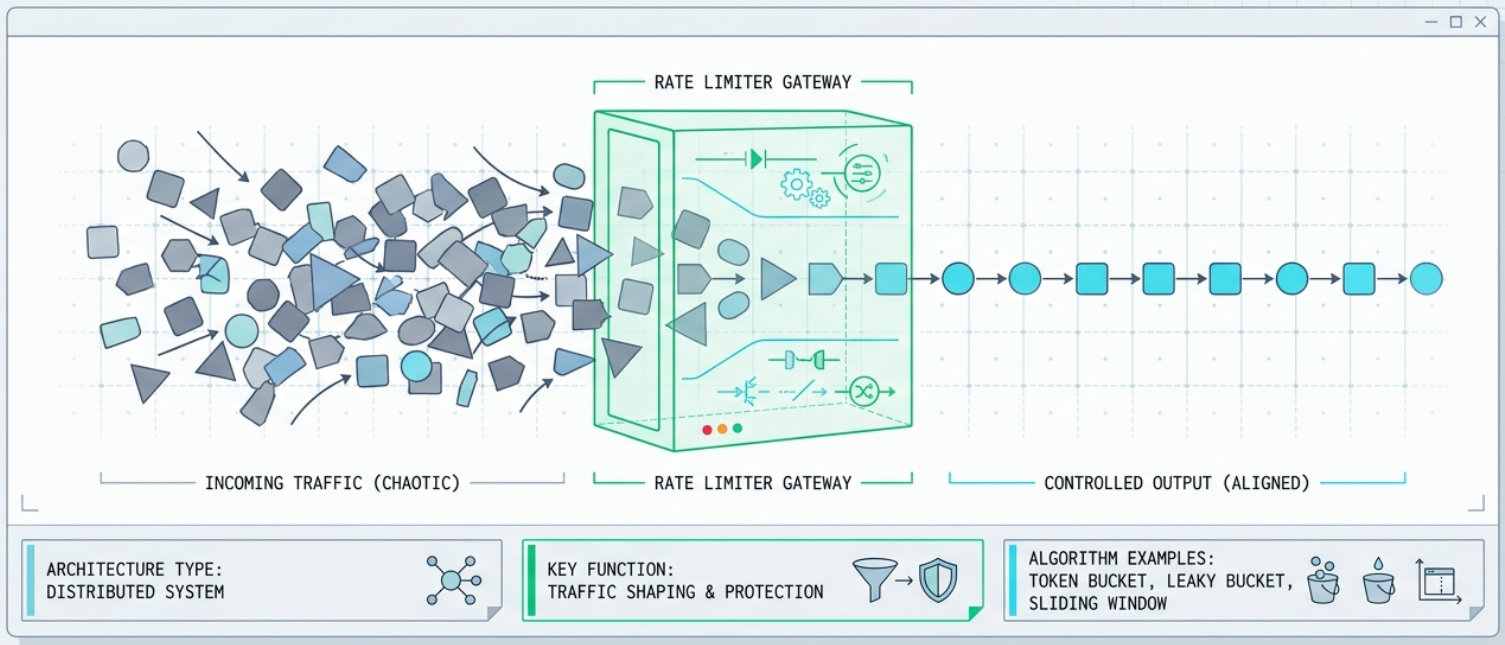


SYSTEM DESIGN: THE RATE LIMITER

THE ARCHITECT'S DEEP-DIVE FOR INTERVIEW PREPARATION



The Defensive Layer: Why We Rate Limit

Prevent Resource Starvation

Defends against application-layer abuse: **DoS attacks**, **brute-force** attempts, and **friendly-fire** misconfigurations.



Manage Policies & Quotas

Ensures fair resource usage in **multi-tenant environments** by enforcing **time-duration** or **allocated constraints**.



Control Data Flow

Regulates **traffic** to prevent **overloading** specific machines, enabling perfectly even **workload distribution**.



Avoid Excess Costs

Caps **billable usage** in **cloud environments** and halts **runaway experiments** or **freemium tier abuse**.



Step 1: Scoping System Requirements

[] System Config

Functional Requirements

- Limit requests per specific time window
- Ensure request limits are entirely configurable
- **Notify** clients upon threshold crossing (HTTP errors)

[] System Constraints

Non-Functional Requirements

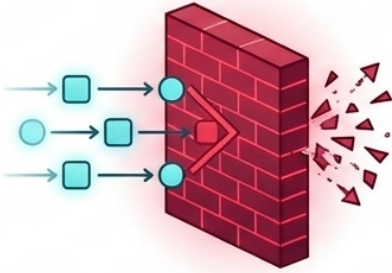
- **High Availability:** Must protect the system without becoming a single point of failure.
- **Low Latency:** Critical path checks must not degrade the end-user experience.
- **Scalability:** Design must support massive and continuous client and request scaling.

Throttling Typology Matrix

[] Hard Throttling

Enforces a strict, unyielding limit.

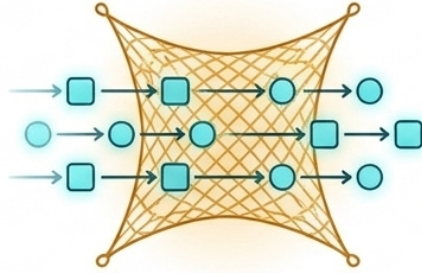
Action: Any request exceeding the threshold is immediately discarded.



[] Soft Throttling

Allows requests to exceed the limit by a specific, configured percentage.

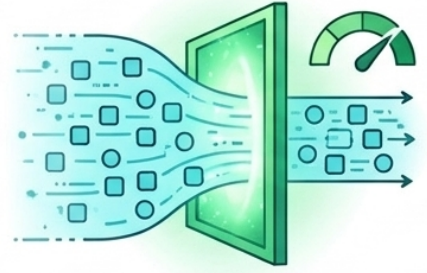
Action: e.g., A limit of 500 with a 5% buffer allows up to 525 requests.



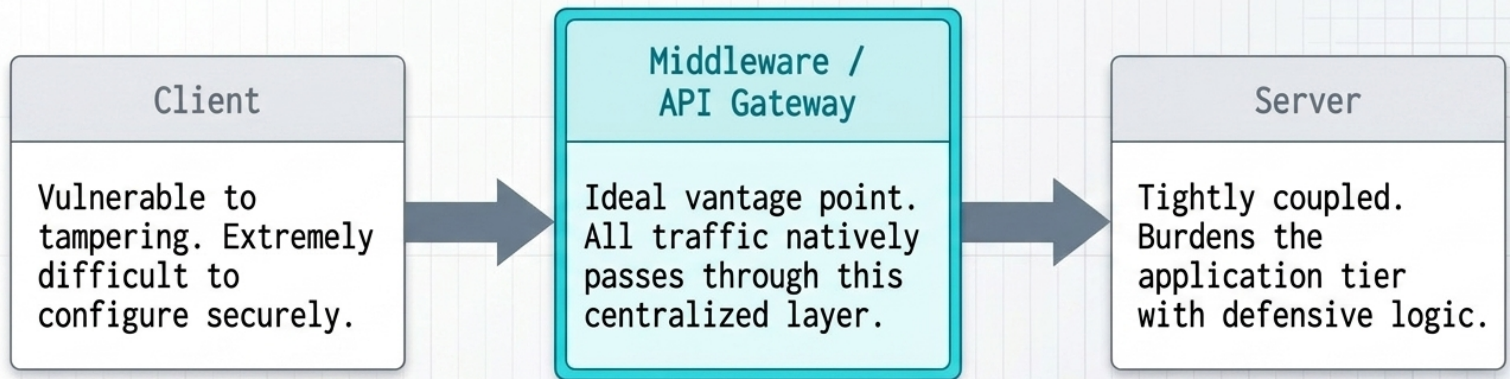
[] Elastic Throttling

Allows traffic to exceed the limit based on external conditions without a fixed cap.

Action: Permits sudden traffic spikes only if the backend system has free, unutilized resources.



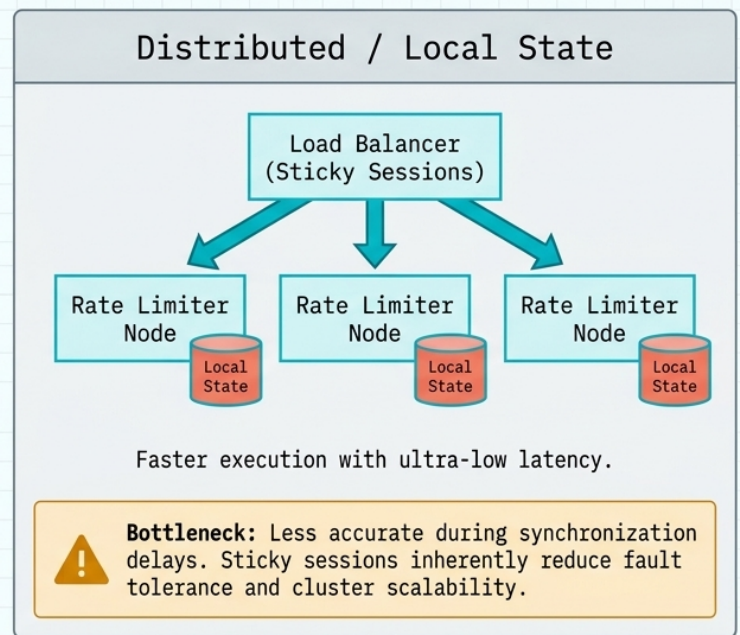
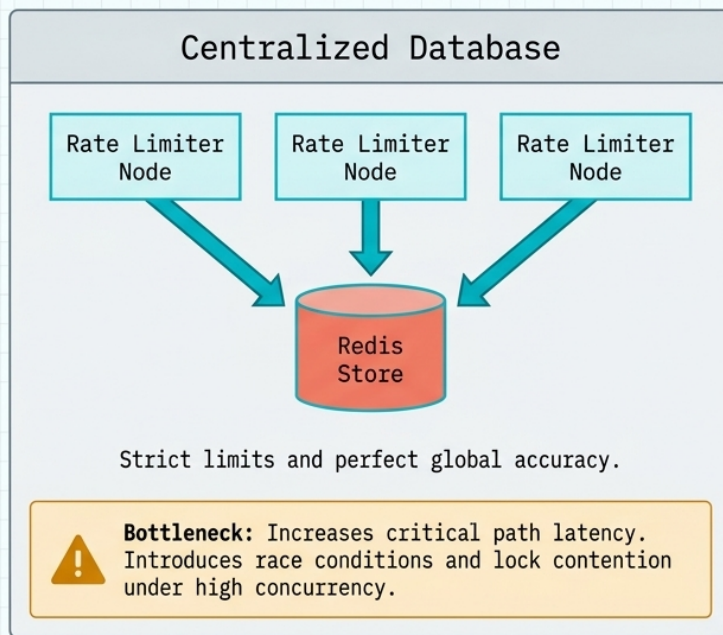
Step 2: Architectural Placement Strategies



Architect's Note:

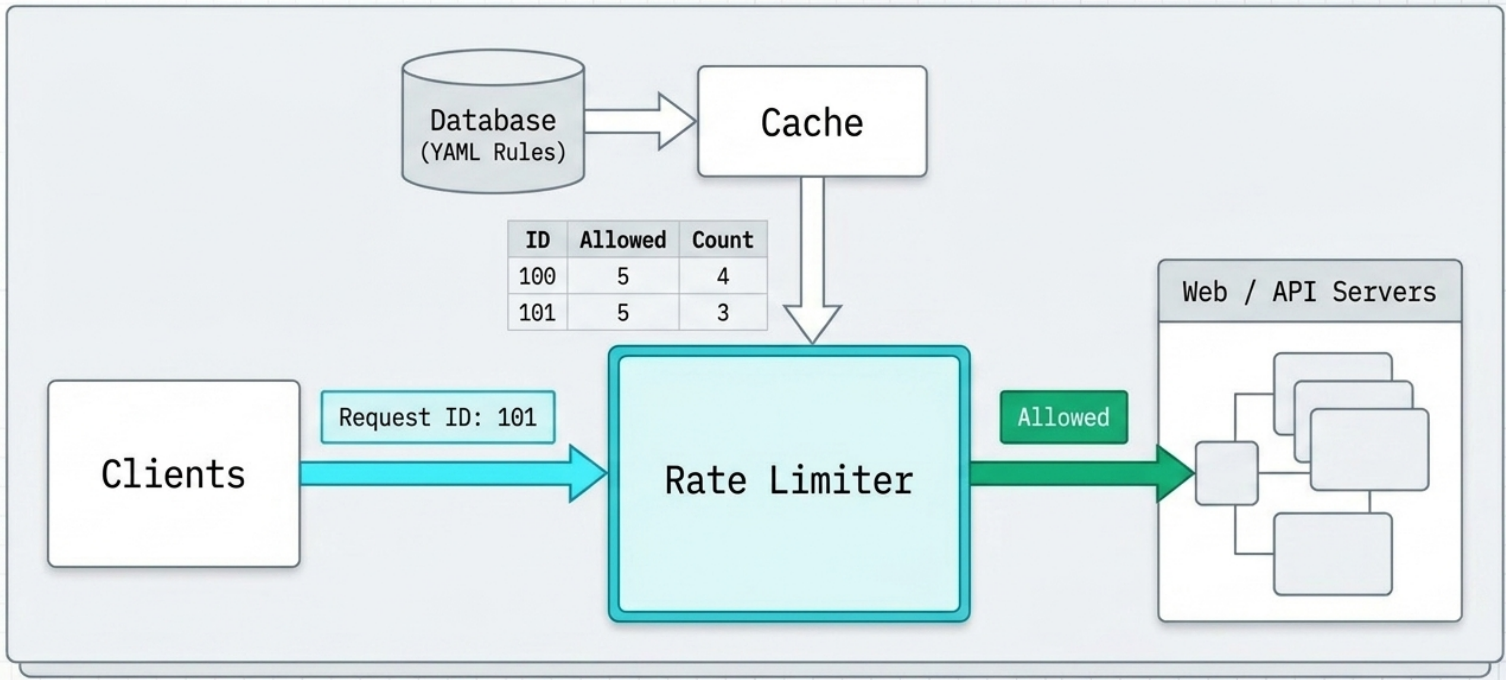
Placement is subjective based on tech stack, but the API Gateway / Middleware layer is the industry standard for decoupled, secure throttling.

State Management at Scale



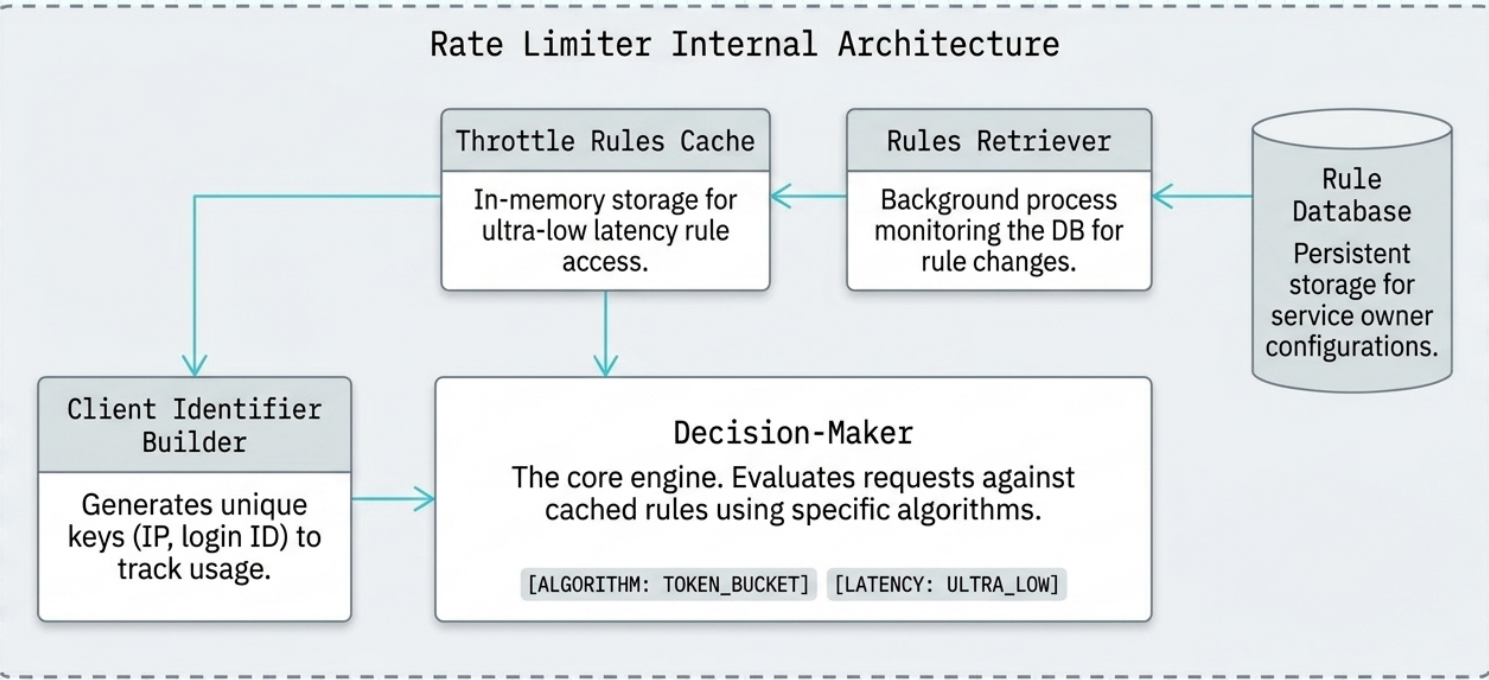
Design Decision: Choose **global counters** vs. individual **per-user counters** based strictly on the specific business quota rules.

Step 3: Component Design & Request Flow

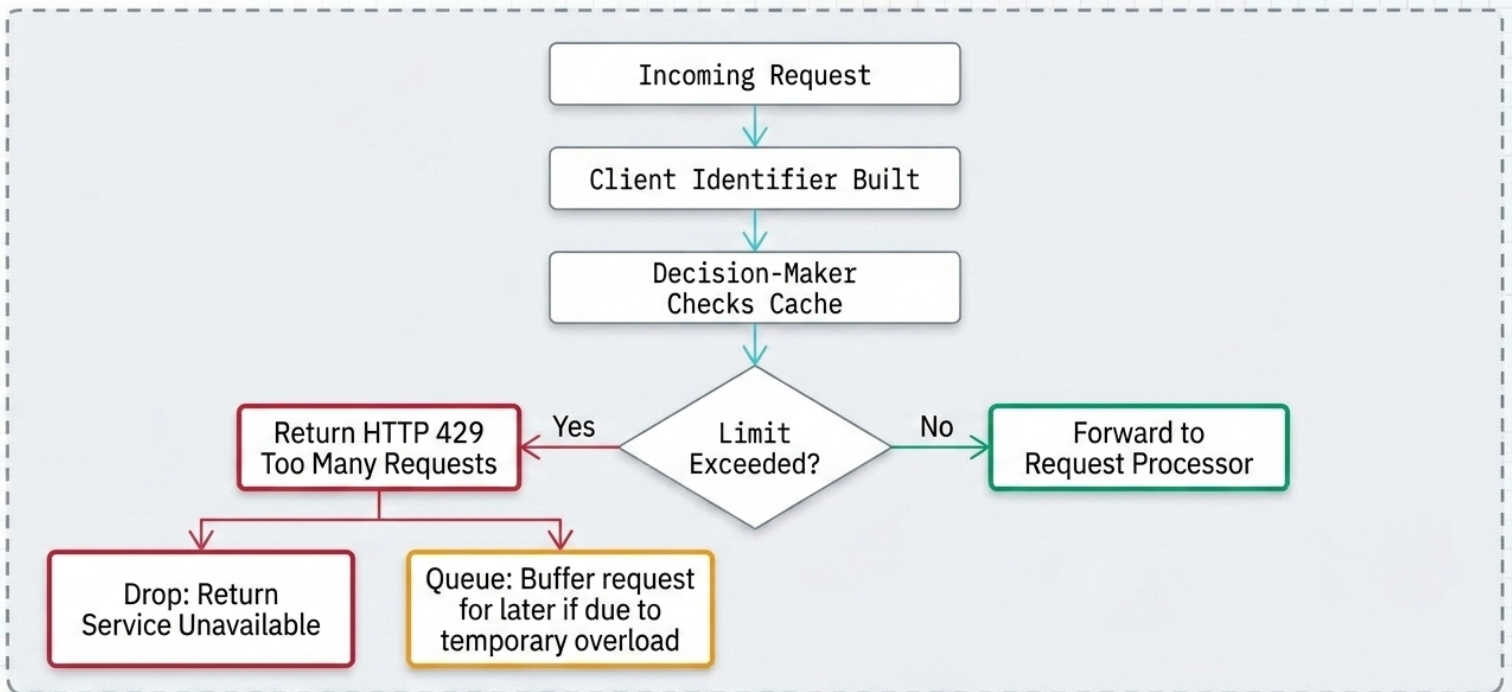


Detailed Component Blueprint

Rate Limiter Internal Architecture



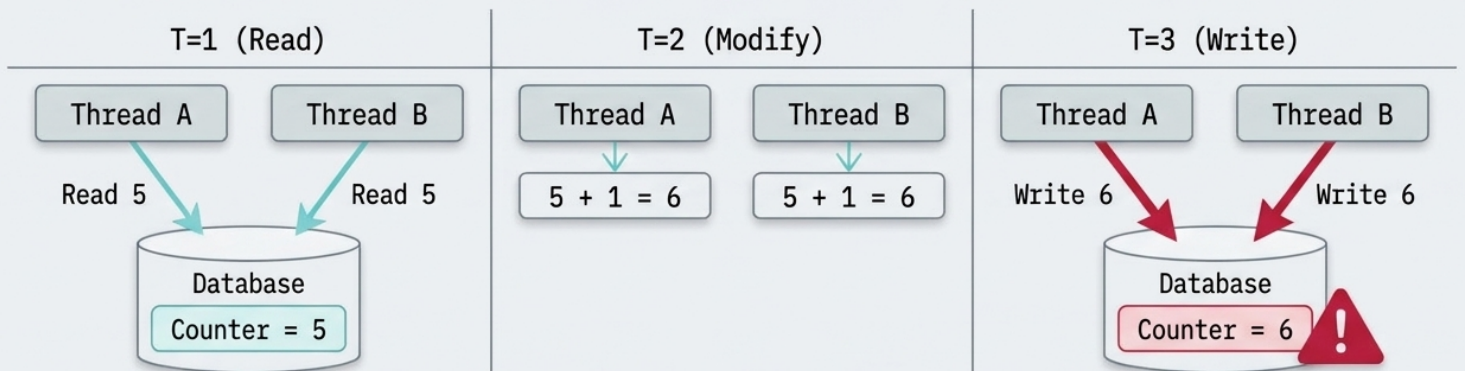
The Request Lifecycle Flowchart



Step 4: Overcoming Bottlenecks

Bottleneck 1: Race Conditions

Read-Modify-Write Failure Timeline



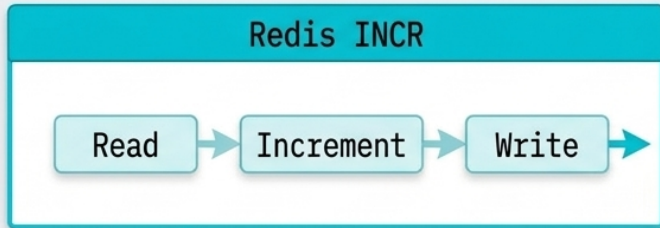
The Undercounting Problem

Without locking, concurrent requests reading the counter state simultaneously will overwrite each other, allowing traffic surges far beyond the hard limit.

Architect's Note: Traditional database locking prevents this but creates catastrophic performance bottlenecks on the critical path.

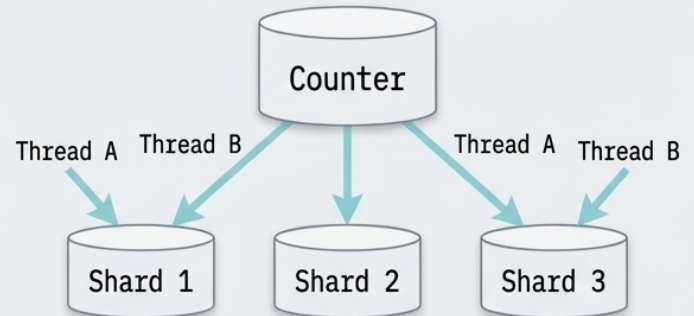
Mitigation: Lock-Free Scaling

Atomic Operations



Increment and return the exact value in a single, un-interruptible, lock-free step.

Sharded Counters



Distributes massive write contention across multiple keys to eliminate collision.



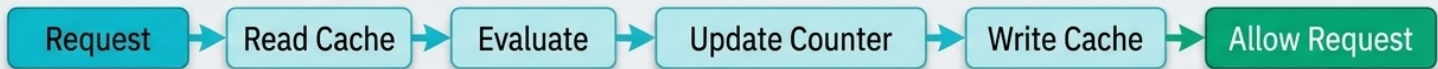
Trade-off Warning: Sharded counters drastically reduce write contention, but reading the total count requires summing all shards, which slightly increases read latency.

Bottleneck 2: Critical Path Latency

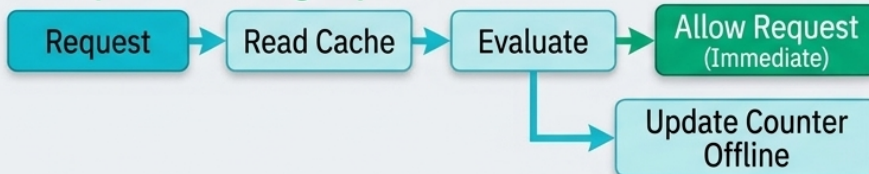
Optimizing Request Throughput with Asynchronous Operations

Critical Path Bottleneck

Synchronous Processing - High Latency



Decoupled Processing - Optimized



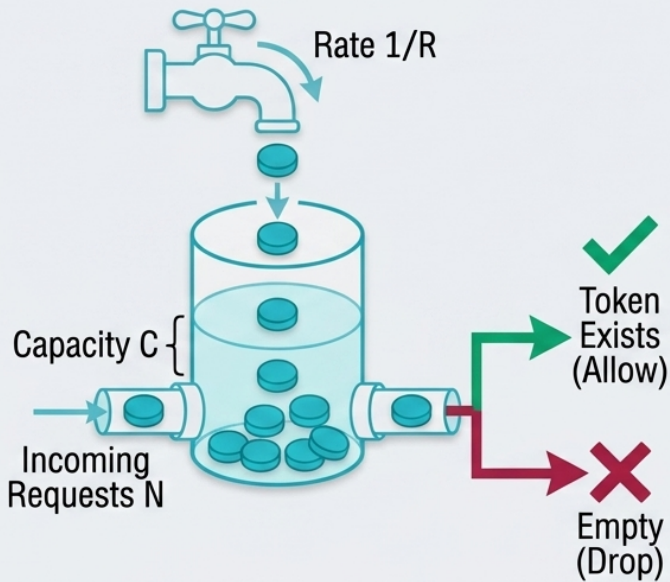
```
If (Request(ID).Count < Max.Limit) {  
    Request(ID).Allowed = True // Online signal  
    Request.Count++           // Offline async update  
}
```

Architect's Connection:
TCP flow control uses a similar mechanism, throttling the sender by advertising a window size asynchronously.
Trade-off: Introduces eventual consistency.

Step 5: Algorithmic Engine

Algorithm 1: Token Bucket

TOKEN BUCKET SCHEMATIC



PARAMETERS

Bucket Capacity (C): Maximum tokens.
Refill Rate (R): Interval of new tokens.
Requests Count (N): Incoming requests.

PROS/CONS MATRIX

Advantages:

- ▶ Highly memory efficient (minimal state requirements).
- ▶ Allows traffic bursts smoothly as long as tokens exist.

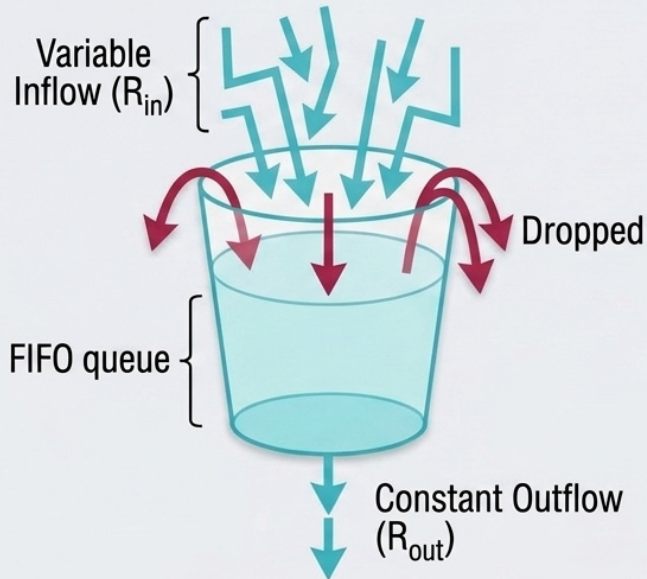
Disadvantages:

- ▶ Tuning the C and R parameters for optimal flow is notoriously challenging in production.

Algorithm 2: Leaking Bucket

Algorithm 2: Leaking Bucket

LEAKING BUCKET SCHEMATIC



PARAMETERS

Queue Capacity (C): Maximum queue size.
Inflow Rate (R_{in}): Variable arrival rate.
Outflow Rate (R_{out}): Constant processing rate.

PROS/CONS MATRIX

Advantages:

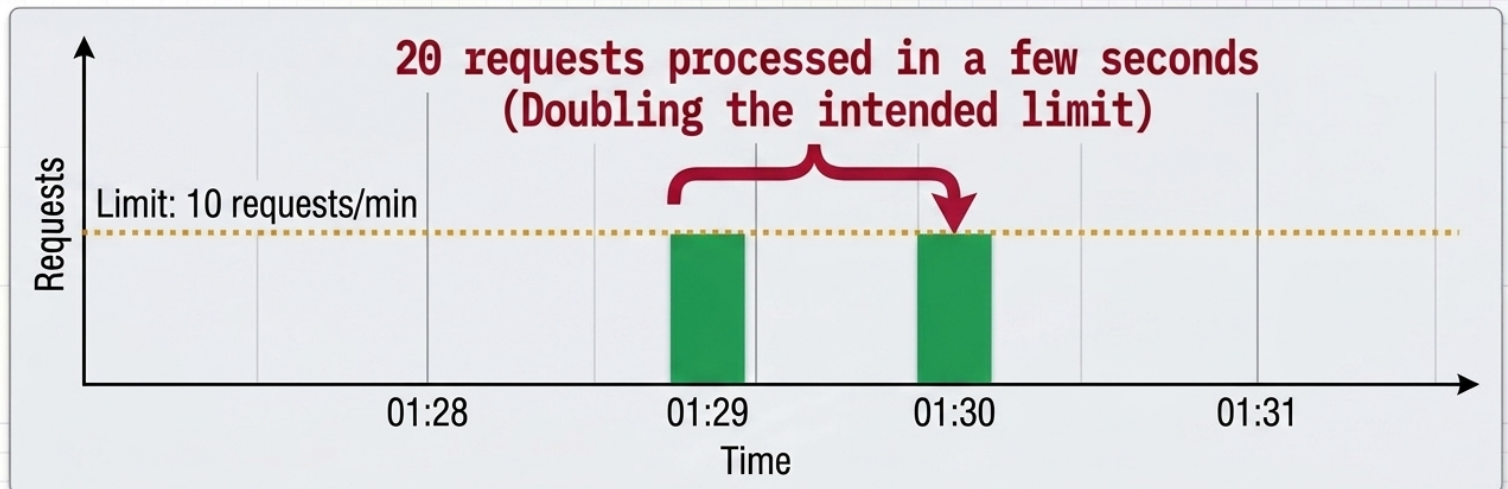
- ▶ Smooths traffic perfectly. Constant outflow prevents bursts entirely.
- ▶ Highly memory efficient state management.

Disadvantages:

- ▶ Fast bursts instantly fill the capacity, causing subsequent critical requests to be discarded while the queue slowly drains.

Algorithm 3: Fixed Window Counter

Algorithm 3: Fixed Window Counter



Parameters

Window Size (W),
Rate Limit (R).

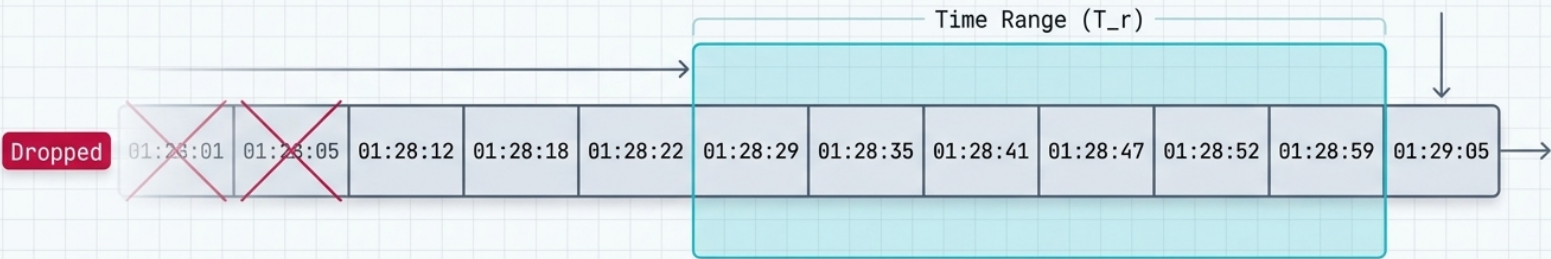
The Fatal Flaw: The Edge Case Burst.

Traffic spikes concentrated exactly at the window boundaries will temporarily overwhelm system capacity.

Algorithm 4: Sliding Window Log

Parameters

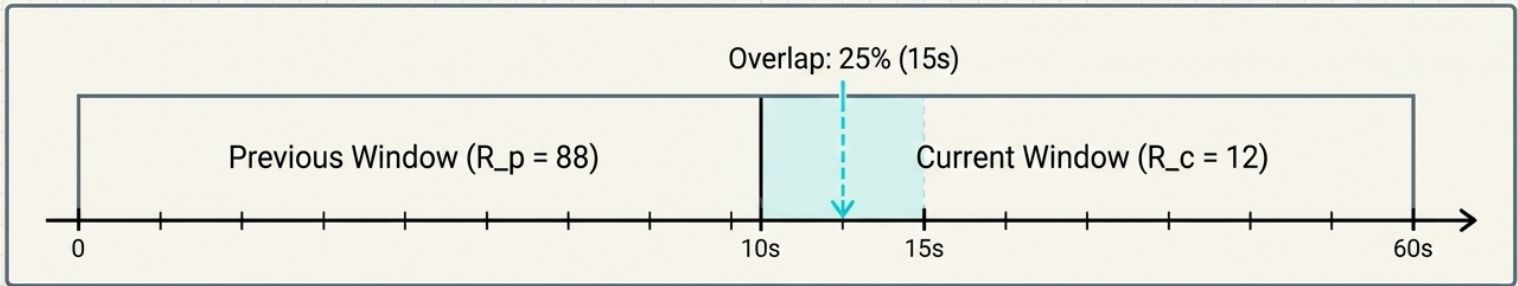
Log Size (L): Max requests allowed.
Time Range (T_n): Duration of the window.



PROS/CONS MATRIX	
Advantages: - Perfect accuracy. Completely eliminates the fixed window boundary burst vulnerability.	Disadvantages: - Extremely high memory footprint. It must persistently store timestamps for every single request (even rejected ones). - Fundamentally unviable as a solution at FAANG-level massive distributed scale.

Algorithm 5: Sliding Window Counter

The Highly Scalable Hybrid Solution



$$\text{Rate} = (R_p * ((\text{time_frame} - \text{overlap_time}) / \text{time_frame})) + R_c$$

Example Execution: $88 * ((60 - 15) / 60) + 12 = 78$

Advantages:

Solves the boundary problem of the Fixed Window without the catastrophic memory cost of the Sliding Log (tracks counts, not individual timestamps).

Disadvantages:

Relies on approximation, assuming requests in the previous window were evenly distributed.

Synthesis: The Architect's Decision Matrix

Algorithm	Space Efficient?	Allows Bursts?	Architect's Verdict
Token Bucket	Yes	Yes	Industry standard for generic API rate limiting (allows initial speed, caps sustained usage).
Leaking Bucket	Yes	No	Use when the backend requires strict, perfectly steady asynchronous processing rates.
Fixed Window	Yes	Yes (at edges)	Use only for non-critical, low-traffic internal APIs due to edge-case flaws.
Sliding Window Log	No	Yes	Mathematically perfect, but memory costs make it an anti-pattern at massive scale.
Sliding Window Counter	Yes	Yes (smoothed)	The optimal, highly scalable hybrid when accurate boundary-smoothing is strictly required.